



武汉理工大学
WUHAN UNIVERSITY OF TECHNOLOGY

企业级应用软件设计与开发

Spec2Vision: 基于 Visual Spec 的 多 Agent 视觉内容生成系统

姓名：朱胜豪

学号：2025303052

班级：研 254

方向：方向一：Agentic AI 原生开发

专业：软件工程

老师：戚欣

2026 年 6 月 22 号

一、选题背景与设计思想

1.1 选题背景

随着大语言模型和多模态生成模型的发展，AI 的能力已经从文本生成扩展到图像、图表、海报、PPT 视觉元素等多模态内容生成场景。在课程汇报、科研展示、产品宣传和日常办公中，用户经常需要快速生成一张视觉资产，例如电商促销主图、论文方法流程图、PPT 封面或信息图。传统方式通常依赖 Photoshop、Canva、PPT 等工具进行手动制作，对设计经验和时间成本都有较高要求。

AI 图像生成工具降低了视觉创作门槛，但真实使用中仍然存在明显痛点：很多用户并不是专业设计师，也不熟悉 Prompt Engineering。他们通常只能表达一个大致意图，例如“做一张科技感 PPT 封面”“帮我画一个论文方法图”“做一张促销海报”。这些表达对人类设计师来说可以进一步沟通确认，但对自动生成系统来说，仍然缺少目标场景、画面比例、视觉风格、关键元素、文案信息和输出格式等关键约束。

更重要的是，Prompt 本身具有天然的语义模糊性。“科技感”既可以被理解为蓝紫色赛博风，也可以被理解为简洁商务科技风；“论文方法图”可能指模型结构图、实验流程图，也可能指数据处理 pipeline；“促销海报”则需要明确商品类别、目标平台、促销文案和视觉风格。如果系统不进一步澄清这些信息，而是直接把 Prompt 交给图像模型，生成结果就很容易偏离用户真实意图。

因此，本项目选择“从模糊需求到结构化视觉资产生成”作为研究和实现主题，设计并实现 Spec2Vision：一个基于 Visual Spec 的多 Agent 视觉内容生成系统。系统的目标不是简单封装图像生成 API，而是在生成之前先理解需求、识别任务类型、主动澄清模糊信息，并将自然语言需求转化为结构化规格，再进行生成、评估和追踪。

图 1-1 从模糊需求到结构化视觉生成任务

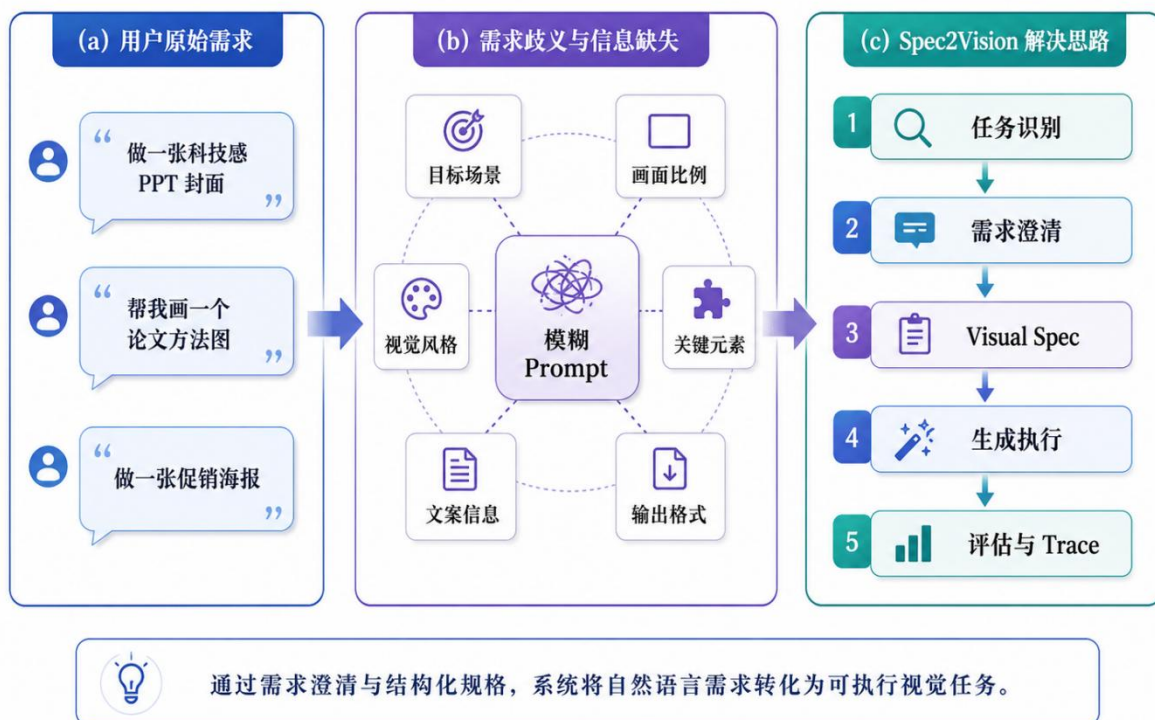


图 1-1 从模糊需求到结构化视觉生成任务

1.2 问题定义

本项目关注的核心问题可以定义为：如何将用户的一句模糊视觉需求，转化为结构化、可执行、可评估、可追踪的视觉生成任务，并最终生成符合场景要求的视觉资产。这个问题的关键不只是“能不能生成图片”，而是“系统是否真正理解了用户想要什么”。

在传统图像生成流程中，用户输入 Prompt，模型直接生成图像。这个流程表面上很简单，但实际上把需求表达压力全部放在了用户身上。用户需要自己想清楚画面比例、主体元素、构图方式、风格、颜色、文字、输出格式等细节。如果用户不具备设计经验或 Prompt 编写经验，生成效果就会高度依赖试错。

因此，Spec2Vision 将问题拆分为四个子任务：首先，系统需要识别用户输入属于哪一类视觉任务，例如电商主图、学术图示或 PPT 视觉设计；其次，系统需要判断需求是否完整，并在信息不足时提出澄清问题；第三，系统需要将自然语言需求转化为结构化 Visual Spec；最后，系统需要记录生成过程并对结果进行评估，使输出不再只是一个黑盒结果。

1.3 现有方案不足

当前常见方案主要包括三类：直接使用通用 AI 生图工具、手动编写复杂 Prompt、以及简单封装图像生成 API 的应用。这些方式虽然都能生成图像，但在复杂视觉任务中仍然存在不足。

第一，通用 AI 生图工具通常采用“一次输入，一次输出”的模式。用户输入一句 Prompt 后，系统直接调用图像模型生成结果。对于简单创意图片，这种方式比较方便；但对于电商主图、学术流程图、PPT 封面这类有明确场景要求的任务来说，单次 Prompt 很难完整表达需求。如果用户没有写清楚比例、结构、主体、文字和风格，系统也不会主动追问，结果偏差就会比较明显。

第二，手动编写 Prompt 的方式依赖用户经验。专业用户可能知道如何描述镜头、构图、色彩、材质、排版和风格，但普通用户往往只会使用日常语言表达需求，例如“好看一点”“正式一点”“像汇报用的”。这些表达虽然自然，但缺少明确的生成指导性，用户需要反复修改 Prompt 和重新生成才能逐渐接近目标效果。

第三，简单封装图像 API 的应用通常只关注“输入文本，输出图片”，缺少系统化的软件设计。它们往往没有明确的需求规格、架构设计、Agent 分工、过程追踪和评估机制。当生成效果不理想时，用户很难判断问题出在任务理解、Prompt 构造、模型能力还是评估标准上。

图 1-2 传统 Prompt 生图方式与 Spec2Vision 对比

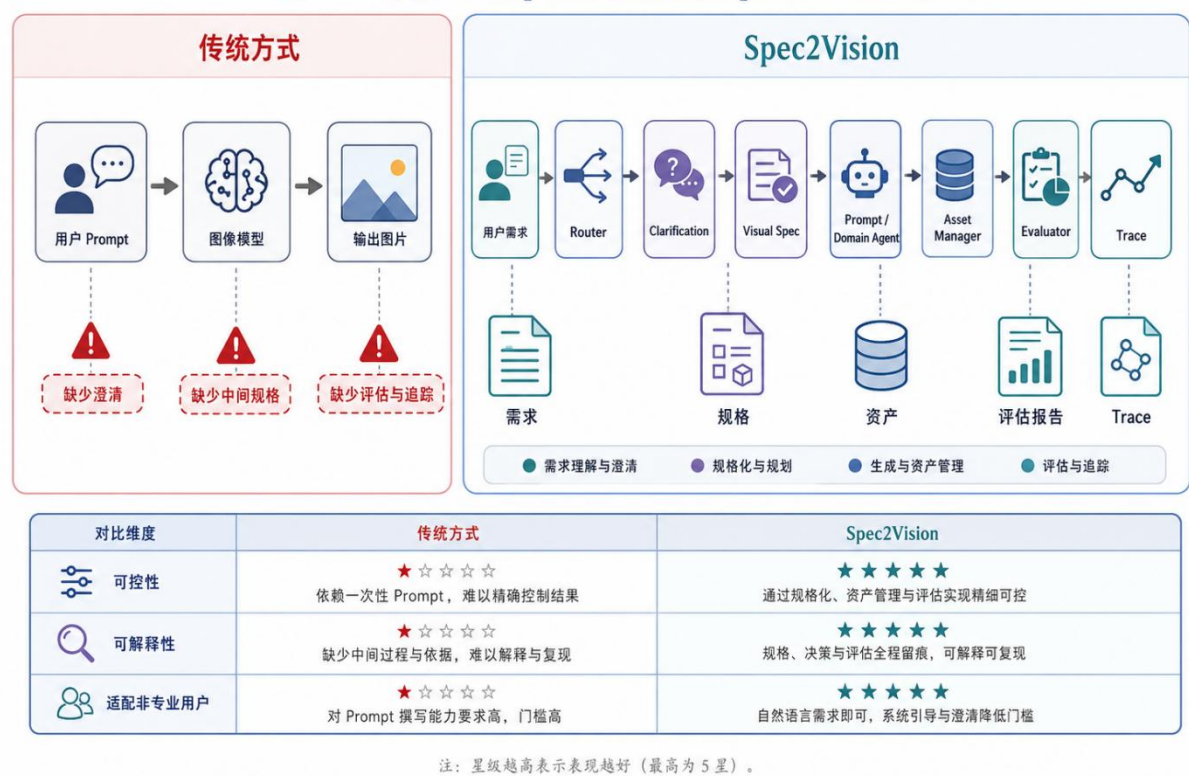


图 1-2 传统 Prompt 生图方式与 Spec2Vision 对比

1.4 项目价值与设计思想

Spec2Vision 的项目价值首先体现在降低使用门槛。系统不要求用户掌握复杂 Prompt 技巧，也不要求用户一次性描述所有设计细节。用户可以先用自然语言表达初步需求，系统再根据任务类型和信息完整度进行分析，并通过澄清问题引导用户补充关键信息。这种方式更适合非专业用户，也更符合真实使用场景。

其次，项目提高了视觉生成任务的可控性。系统通过 Visual Spec 将需求转化为结构化字段，例如任务类型、目标场景、画面比例、关键元素、风格约束、输出格式和评估标准。相比直接使用自然语言 Prompt，结构化规格更容易被程序读取、校验和复用，也更适合在多个 Agent 之间传递。

第三，项目提高了生成过程的可解释性。Spec2Vision 记录完整 Agent Trace，包括任务路由、澄清结果、Visual Spec、Prompt、Provider、生成结果和评估报告。用户和评审者可以回溯系统为什么这样生成，而不仅仅看到最终图片。

本项目的核心设计思想可以概括为：以 Visual Spec 作为中间层，以多 Agent 协作为执行机制，以 Trace 和 Evaluation 作为质量保障。Visual Spec 负责把模糊自然

语言变成可执行规格，多 Agent 负责把复杂任务拆分为可维护步骤，Trace 和 Evaluation 则使系统具备可观察和可评估能力。

表 1-1 Spec2Vision 项目价值与设计对应关系

设计目标	对应机制	体现价值
降低 Prompt 门槛	Router + Clarification	帮助非专业用户补全需求
提高生成可控性	Visual Spec 结构化字段	让生成过程有明确执行依据
增强可解释性	Agent Trace	记录中间决策和执行过程
保证结果质量	Evaluator + Rubric	对输出结果进行多维度评估
形成工程闭环	Specs + API + Tests	体现从设计到实现再到验证的完整流程

1.5 技术路线

本项目采用“前端交互 + 后端 API + 多 Agent 编排 + 生成工具 + 评估模块”的技术路线。前端层使用 Streamlit 构建交互界面，支持用户输入需求、查看 Visual Spec、预览 PNG / SVG 结果，并查看评估报告和 Agent Trace。后端层使用 FastAPI 提供 /generate、/clarify、/extract、/tasks 等接口，负责请求校验、任务管理和结果返回。

在 Agent 编排层，项目使用 LangGraph / pipeline 组织多个 Agent 的执行流程。核心 Agent 包括 TaskRouterAgent、ClarificationAgent、RequirementAgent、VisualSpecAgent、Domain Agent、PromptAgent、AssetManagerAgent 和 Evaluator。每个 Agent 负责一个明确步骤，并通过结构化状态传递信息。

在生成与扩展能力方面，系统支持 Mock Provider 和 OpenAI Images API。Mock Provider 可以在无 API Key 的情况下离线运行，保证测试和课程验收可复现；OpenAI Images API 可用于展示真实图像生成效果。系统同时集成 Agentic RAG 和 MCP Tool Server，使视觉规范检索和外部工具调用具备扩展空间。

1.6 本章小结

综上，Spec2Vision 的选题并不是简单地封装一个图像生成 API，而是围绕真实视觉生成任务中的用户不专业、Prompt 难编写、表达模糊、过程黑盒和结果难评估等问题，设计一个具备需求澄清、规格生成、多 Agent 协作、过程追踪和结果评估能力的 Agentic AI 原生系统。

本章从选题背景、问题定义、现有方案不足、项目价值、设计思想和技术路线几个方面说明了项目的设计逻辑。整体来看，本项目的核心价值在于将“用户的一句话需求”转化为“可执行的 Visual Spec”，再通过多 Agent 协作完成生成和评估。

二、Specs 规格文档

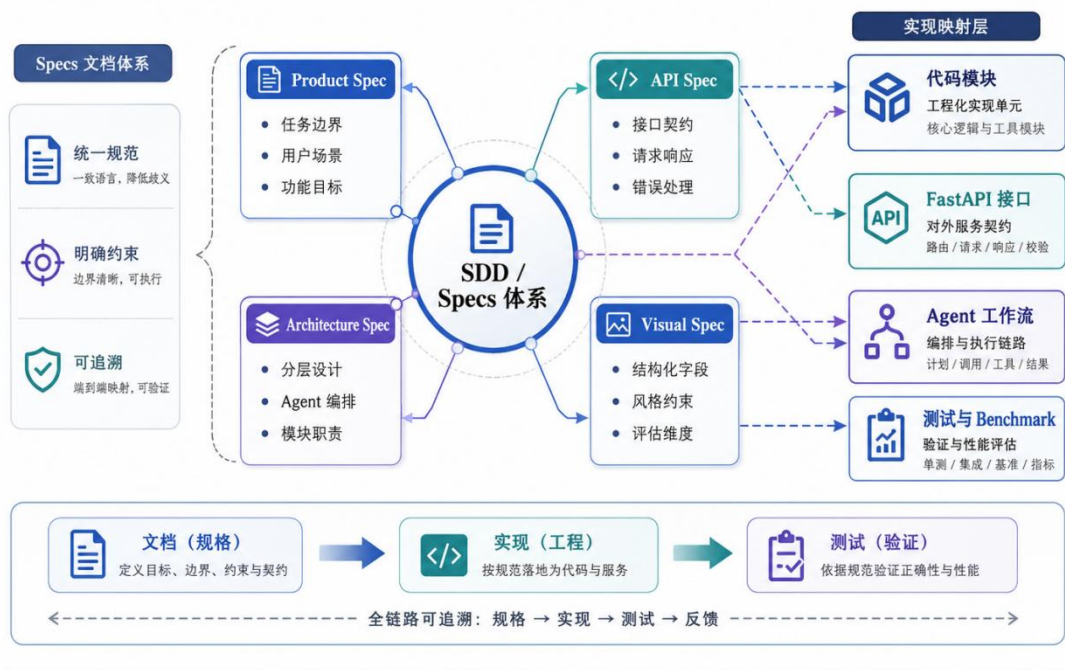
2.1 SDD 方法与规格文档组织

本章围绕课程要求中的 Specs 规格文档展开，重点说明 Spec2Vision 在产品 设计、架构设计、接口设计和视觉生成规格设计中的规格化思路。与“先写代码、后补 文档”的开发方式不同，本项目采用 SDD（Specification-Driven Development，规格驱 动开发）的思想：先明确系统要解决的问题、用户需要什么、模块之间如何协作、接 口如何交互，再根据规格实现代码、测试和 Demo。

在 Spec2Vision 中，规格文档不是附属说明，而是系统设计和实现的依据。 Product Spec 用来限定产品目标、用户场景、核心功能和范围边界；Architecture Spec 用来描述系统分层、Agent 交互流程和数据流；API Spec 用来约定前后端和外部调 用方之间的接口契约；Visual Spec 则是本项目特有的核心中间结构，用于把用户自 然语言需求转化为可执行的视觉生成规格。

通过这种文档组织方式，系统开发过程可以保持较高的一致性。需求层面的目标 会映射到 Product Spec；模块和流程会映射到 Architecture Spec；接口输入输出会映 射到 API Spec；具体生成任务会映射到 Visual Spec。最终，测试用例和 Demo 也会 反向验证这些规格是否真正被实现。

图 2-1 Specs 文档体系与实现映射



规格文档不仅描述系统，还直接约束实现方式与测试验证路径。

图 2-1 Specs 文档体系与实现映射

2.2 Product Spec: 产品目标与功能边界

Product Spec 主要回答“系统要为谁解决什么问题”。Spec2Vision 面向需要快速生成视觉资产、但不一定熟悉设计工具和 Prompt 写法的普通用户。例如，学生可能需要为课程汇报生成一张 PPT 封面；科研人员可能需要把论文方法整理成流程图；电商运营人员可能需要快速生成促销主图。这些用户的共同特点是：他们知道大致目标，但未必能一次性写出完整、清晰、专业的 Prompt。

因此，Product Spec 将系统目标定义为：降低非专业用户的视觉生成门槛，把模糊自然语言需求转化为结构化、可执行、可评估的视觉生成任务。系统不要求用户从一开始就完整描述所有细节，而是通过任务路由和澄清机制逐步补全需求。

在功能范围上，Spec2Vision 支持三类核心任务：电商主图、学术图示和 PPT 视觉设计。选择这三类任务的原因是它们具有不同的视觉目标和约束条件，能够较好体现系统的领域适配能力。电商主图强调视觉吸引力、商品主体和促销信息；学术图示强调结构清晰、逻辑关系和术语准确；PPT 视觉设计强调主题表达、版式平衡和演示氛围。

表 2-1 Product Spec 核心功能范围

功能模块	功能说明	价值
任务路由	识别用户输入属于电商、学术或 PPT 视觉任务	为后续生成选择合适策略
需求澄清	在信息不足或语义模糊时提出澄清问题	降低非专业用户编写 Prompt 的压力
Visual Spec 生成	将自然语言需求转化为结构化规格	提高生成过程的可控性
视觉资产生成	调用 Mock Provider、OpenAI Images API 或 SVG 生成器	输出可下载的 PNG 或 SVG 资产
结果评估	从格式、尺寸、视觉指标和 Rubric 角度评估结果	使生成结果可解释、可比较
Trace 记录	记录 Agent 执行步骤、耗时、Provider 和中间结果	提高系统可追踪性和可调试性

2.3 Visual Spec: 从模糊 Prompt 到结构化规格

Visual Spec 是 Spec2Vision 中最重要的中间层。普通用户输入的是自然语言 Prompt，而生成模型真正需要的是更明确的任务描述。Visual Spec 的作用就是把模糊的自然语言转化为结构化字段，使系统可以明确知道要生成什么类型的视觉资产、

使用什么比例、包含哪些关键元素、遵守哪些约束。

例如，用户输入“帮我做一张科技感 PPT 封面”时，系统不能直接假设所有细节。它需要进一步判断这是 PPT 视觉任务，需要补充主题、比例、色彩、文字内容和风格要求。完成澄清后，系统会将这些信息整理成 Visual Spec。这样后续 Prompt Agent 和 Asset Manager Agent 不再直接依赖原始 Prompt，而是依赖更稳定的结构化规格。

Visual Spec 的设计体现了 SDD 中“规格可执行”的要求。它不是单纯的文字说明，而是可以被代码读取、传递和校验的数据结构。系统可以根据 task_type 选择领域 Agent，根据 aspect_ratio 计算输出尺寸，根据 key_elements 构造 Prompt，也可以根据 constraints 和 evaluation_hints 判断生成结果是否满足要求。

表 2-2 Visual Spec 关键字段设计

字段	含义	示例
task_type	任务类型，用于区分电商、学术、PPT 等场景	ppt_visual
title	视觉资产标题或主题	智能驱动的未来架构
aspect_ratio	输出比例或尺寸要求	16:9
key_elements	画面中必须出现的核心元素	标题文字、科技背景、抽象线条
style	视觉风格要求	简洁、科技感、蓝紫色
constraints	生成时需要遵守的限制条件	文字清晰、适合课堂展示、避免过度拥挤
output_format	输出格式	PNG / SVG
evaluation_hints	后续评估时关注的指标	可读性、主题一致性、构图完整性

2.4 Architecture Spec 与 API Spec

Architecture Spec 主要回答“系统如何组织模块，以及模块之间如何协作”。Spec2Vision 采用分层架构设计，从上到下可以划分为 UI 层、API 层、Service 层、Agent 层、Tool 层、Storage 层和 Evaluation 层。前端通过 Streamlit 提供交互界面，后端通过 FastAPI 提供接口，Service 层负责组织业务流程，Agent 层负责执行任务理解和生成逻辑，Tool 层负责图像生成、文档解析、RAG 检索和评估工具调用。

在 Agent 设计上，系统没有让单个大模型完成所有任务，而是将视觉生成流程拆分为多个职责明确的 Agent。TaskRouterAgent 负责识别任务类型；ClarificationAgent 负责判断是否需要追问；RequirementAgent 负责整理需求；VisualSpecAgent 负责生成结构化规格；Domain Agent 负责根据任务类型进行领域增

强; PromptAgent 负责构造最终 Prompt; AssetManagerAgent 负责调用生成 Provider; Evaluator 负责结果评估。

API Spec 主要回答“系统能力如何对外暴露”。Spec2Vision 的后端使用 FastAPI 提供接口，前端 UI、测试脚本和外部调用方都可以通过这些接口访问系统能力。API Spec 的作用是明确每个接口的路径、请求参数、响应字段、错误处理和状态变化，避免前后端或不同模块之间产生理解偏差。

表 2-3 API Spec 核心接口设计

方法	路径	功能说明
GET	/health	健康检查，返回系统状态和 Provider 信息
POST	/extract	上传 PDF 或 TXT，抽取文档内容并生成 document_context
POST	/clarify	根据用户输入和任务类型返回澄清问题
POST	/generate	触发完整视觉生成流水线
GET	/tasks	分页获取任务列表
GET	/tasks/{id}	获取任务详情，包括 Visual Spec、评估结果和 Trace
GET	/tasks/{id}/asset	下载生成的 PNG 或 SVG 视觉资产
DELETE	/tasks/{id}	删除指定任务及相关记录

2.5 规格文档与代码实现的一致性

Specs 的价值不仅在于描述系统，更在于指导实现和验证实现。本项目在开发过程中保持规格文档、代码模块和测试用例之间的一致性。Product Spec 中定义的核心功能会在 agents、services、tools 和 API 中落地；Architecture Spec 中定义的分层结构会对应到 app 目录下的不同模块；API Spec 中定义的接口会对应 FastAPI 路由和测试用例；Visual Spec 中定义的数据结构会对应 Pydantic 模型和生成流程中的中间状态。

例如，Product Spec 中强调“需求澄清”，因此代码中设计了 ClarificationAgent，并在低置信度任务下返回澄清问题；Architecture Spec 中强调“多 Agent 协作”，因此代码中将 Router、Spec、Prompt、Asset、Evaluator 分为不同组件；API Spec 中要求任务详情可查询，因此系统提供 /tasks/{id} 接口，并返回任务状态、评估报告和 Trace 信息。

测试与评估同样围绕规格展开。功能测试验证接口是否符合 API Spec；Agent 行为测试验证路由、澄清和规格生成是否符合预期；Benchmark 验证多类任务的端到端

稳定性；Demo 用例则验证从用户输入到最终资产输出的完整闭环。通过这种“规格 - 实现 - 测试”的对应关系，项目可以更清晰地展示 SDD 方法论的应用效果。

表 2-4 规格到实现与测试的映射

规格要求	代码实现位置	验证方式
任务类型识别	app/agents/task_router.py	Router 单元测试、benchmark routing_accuracy
需求澄清	app/agents/clarification_agent.py	澄清逻辑测试、低置信度输入测试
Visual Spec 生成	app/agents/visual_spec_agent.py 与 models	结构字段校验、端到端 Demo
多 Agent 编排	app/graph/visionflow_graph.py	pipeline 测试、trace 检查
图像 / SVG 生成	app/tools 与 provider 相关模块	Mock 生成测试、asset 下载测试
评估报告	app/tools/evaluator.py	Evaluator rubric 测试、API 响应测试

2.6 本章小结

本章围绕 Specs 规格文档说明了 Spec2Vision 的 SDD 设计思路。Product Spec 明确了系统面向非专业用户的视觉生成需求，强调通过澄清和规格化降低 Prompt 编写门槛；Architecture Spec 描述了系统分层、多 Agent 协作和数据流设计；API Spec 固定了系统对外暴露能力的接口契约；Visual Spec 则作为本项目的核心中间层，将模糊自然语言需求转化为可执行的视觉生成规格。

整体来看，Spec2Vision 的规格文档并不是简单说明材料，而是贯穿需求分析、架构设计、代码实现和测试验证的核心依据。通过“规格 - 实现 - 测试”的对应关系，项目能够体现 SDD 方法论中的文档完整性、规格可执行性和工程可追踪性。

三、系统架构与设计

3.1 架构设计目标

本章围绕 Spec2Vision 的系统架构、Agent 交互流程和数据流设计进行说明。根据课程要求，本章重点展示系统如何从用户输入出发，经过前端交互、后端接口、多 Agent 编排、工具调用、资生产成和质量评估，最终形成可下载、可解释、可追踪的视觉生成结果。

Spec2Vision 的架构设计目标主要包括四点。第一，降低非专业用户的使用门槛，系统不能假设用户能够写出完整、准确的 Prompt，而应当通过 Agent 主动识别模糊点并提出澄清。第二，将自然语言需求转化为结构化 Visual Spec，使后续生成步骤拥有明确的执行依据。第三，将复杂任务拆分为多个职责清晰的 Agent，提高系统可维护性和可扩展性。第四，通过 Evaluation Report 和 Trace JSON 记录结果质量与执行过程，使系统不再只是黑盒图像生成工具。

因此，系统整体采用分层架构：上层负责用户交互，中层负责 API 与业务编排，下层通过多 Agent workflows 调用生成工具、评估模块、RAG 知识库和持久化存储。



图 3-1 Spec2Vision 系统总体架构

3.2 系统总体架构

Spec2Vision 的总体架构由 UI 交互层、API 服务层、业务编排层、多 Agent 工作流层、工具与数据层组成。UI 层使用 Streamlit 实现，负责接收用户自然语言需求、展示 Visual Spec、预览生成结果、展示 Rubric 评分和 Agent Trace。API 层使用 FastAPI 实现，对外提供 /generate、/clarify、/extract、/tasks 等接口，统一管理请求输入和响应输出。

业务编排层以 GenerationService 为核心，负责创建任务、调用 Agent 工作流、保存中间状态和汇总最终结果。多 Agent 工作流层是系统的核心部分，负责将用户需求分解为任务路由、需求澄清、规格生成、Prompt 构建、资产生成和结果评估等步骤。工具与数据层封装了 Mock Provider、OpenAI Images API、SVG Diagram Generator、Evaluator、RAG、MCP、SQLite 和文件存储等能力。

这种分层设计使系统的职责边界比较清晰。前端只关注交互展示，后端只关注 API 契约，Agent 层只关注任务推理与状态流转，工具层只关注外部能力调用。不同层之间通过结构化对象进行传递，降低了模块之间的耦合度。

表 3-1 系统分层与职责说明

层级	核心技术 / 模块	主要职责
UI 交互层	Streamlit	需求输入、结果预览、Trace 展示
API 服务层	FastAPI	接口路由、请求校验、响应封装
业务编排层	GenerationService	任务创建、流程调用、状态管理
Agent 工作流层	LangGraph / Pipeline	多 Agent 编排、条件分支、状态传递
工具与数据层	Provider / Evaluator / RAG / MCP / SQLite	图像生成、质量评估、知识增强、工具暴露、数据存储

3.3 多 Agent 协作与数据流设计

Agent 交互流程是本系统区别于普通图像生成应用的核心设计。传统方式通常是“Prompt 输入—模型生成—返回图片”，而 Spec2Vision 将这个流程拆分为多个具有明确职责的 Agent。每个 Agent 只处理一个相对独立的问题，并将结果写入共享状态。这样做可以避免单个模型一次性承担所有推理和生成职责，也便于对每一步进行测试和调试。

具体来说，TaskRouterAgent 首先根据用户输入判断任务类型，例如电商主图、学术图示或 PPT 视觉设计，并输出置信度和判断依据。如果系统发现需求信息不足，

ClarificationAgent 会提出澄清问题，引导用户补充比例、风格、目标平台、文字内容等关键信息。RequirementAgent 和 VisualSpecAgent 会将自然语言需求整理为结构化 Visual Spec。随后，Domain Agent 根据具体场景进行领域增强，PromptAgent 将规格转换为适合生成模型的 Prompt，AssetManagerAgent 调用对应工具生成 PNG 或 SVG，最后 Evaluator 对结果进行检查和评分。

在数据流设计上，Spec2Vision 强调从“自然语言”逐步转化为“结构化对象”。用户最初输入的是一段模糊文本，系统首先生成 RouteResult，用于表示任务类型、置信度和是否需要澄清。澄清阶段会产生用户补充答案，随后这些信息会被整合进 Visual Spec。Visual Spec 是后续生成的核心数据结构，它包含标题、关键元素、风格约束、比例、输出格式和领域扩展字段。

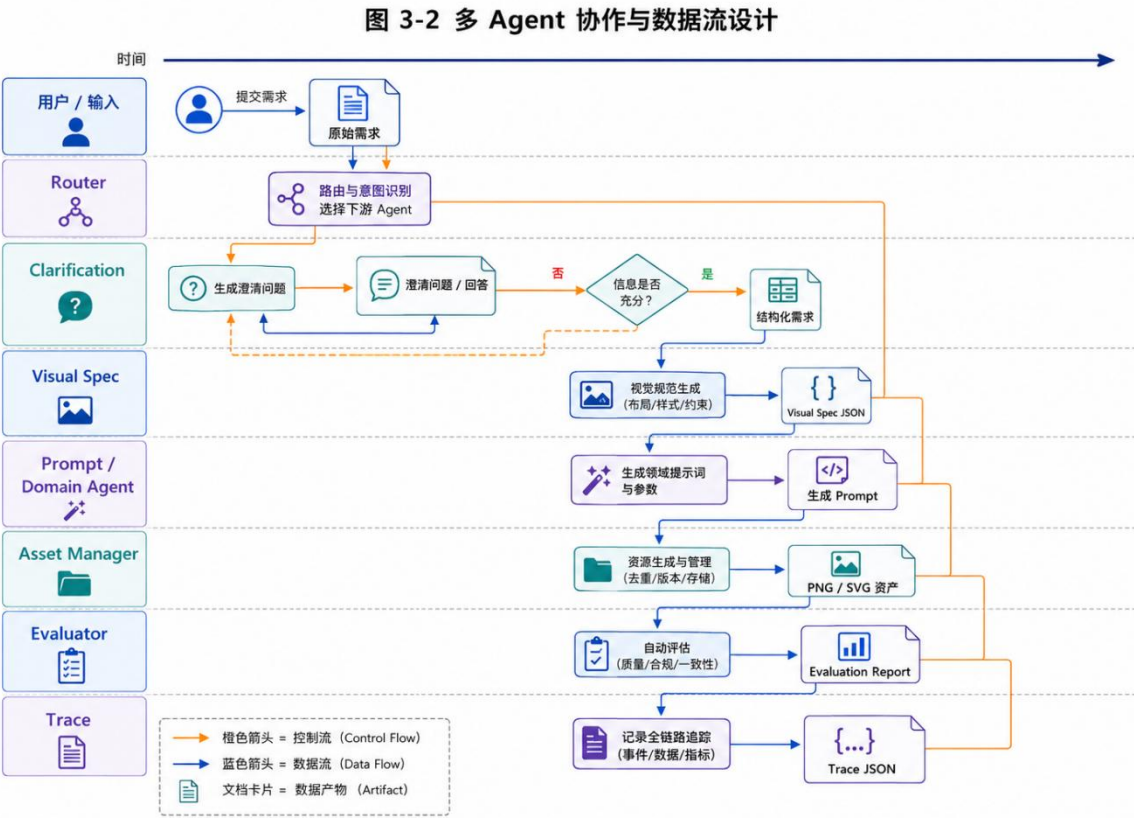


图 3-2 多 Agent 协作与数据流设计

3.4 核心模块职责划分

为了保证系统具有良好的工程可维护性，Spec2Vision 对核心模块进行了明确职责划分。Agent 模块负责推理和状态转换，Tool 模块负责具体能力调用，Service 模块负责业务编排，Model 模块负责数据结构定义，UI 模块负责结果展示。这样的设

计使每个模块都可以独立测试，也方便后续扩展新的任务类型、生成 Provider 或评估方式。

PromptAgent 不直接依赖原始用户输入，而是根据 Visual Spec 构造 PromptPayload。AssetManagerAgent 根据 PromptPayload 和任务类型选择对应的生成工具，输出 PNG、SVG 或 Mock 资产。Evaluator 读取生成资产和 Visual Spec，产生 EvaluationReport。与此同时，每一步执行过程都会写入 TraceJSON，包括 Agent 名称、pipeline_step、耗时、Provider、warnings 和关键元数据。

这种数据流设计的优势在于，中间结果都可以被查看、保存和测试。当结果不符合预期时，可以判断问题是出在任务路由、需求澄清、规格生成、Prompt 构造还是图像生成阶段，而不是只能看到最终图片。

表 3-2 核心 Agent 职责说明

Agent	输入	输出 / 职责
TaskRouterAgent	用户原始需求	判断任务类型、置信度和是否需要澄清
ClarificationAgent	模糊需求与缺失字段	生成澄清问题并接收补充信息
VisualSpecAgent	需求文本与澄清结果	生成结构化 Visual Spec
Domain Agent	Visual Spec 与任务类型	根据电商、学术或 PPT 场景进行领域增强
PromptAgent	Visual Spec	构造可执行 PromptPayload
AssetManagerAgent	PromptPayload 与 Provider 配置	调用图像或 SVG 生成工具
Evaluator	生成资产与 Visual Spec	输出 EvaluationReport 和 Rubric 评分

3.5 Provider 与工具调用设计

在生成工具设计上，系统没有将生成能力固定绑定到某一个外部服务，而是通过 Provider 机制实现可插拔调用。默认情况下，系统使用 Mock Provider 生成离线可复现的 PNG 资产，保证无 API Key 时也可以完成测试和课程验收。当配置 IMAGE_PROVIDER 为 openai 时，系统可以调用 OpenAI Images API 生成真实图像。对于学术流程图任务，系统还支持本地 SVG Diagram Generator，使流程图类任务能够得到更清晰的结构化表达。

除了生成 Provider，系统还包含 Evaluator、RAG 和 MCP 等工具能力。Evaluator 用于对结果进行确定性检查、图像启发式指标分析和 Rubric 评分；Agentic RAG 用

于从本地知识库中检索电商、学术和 PPT 视觉规范，为规格生成和 Prompt 构造提供参考；MCP Tool Server 则将 `create_visual_spec`、`generate_visual_asset`、`evaluate_visual_asset` 和 `query_generation_trace` 等能力暴露给外部 Agent 或工具使用。

3.6 本章小结

本章从系统总体架构、Agent 交互流程、数据流设计、模块职责划分和 Provider 工具调用等方面说明了 Spec2Vision 的系统架构与设计。整体来看，系统采用分层架构降低模块耦合，通过多 Agent 流程完成复杂任务拆解，通过 Visual Spec 实现需求规格化，通过 Trace 和 Evaluation 保证生成过程可观察、结果可评估。

与简单的 Prompt 生图应用相比，Spec2Vision 更强调工程化设计和 Agentic AI 原生开发思路。系统不是直接把用户输入交给模型，而是通过任务识别、澄清、规格生成、生成执行和结果评估形成完整闭环。这种设计既满足课程对系统架构、Agent 交互流程和数据流设计的要求，也为后续扩展新的视觉任务、模型 Provider 和评估方法提供了基础。

四、关键实现与代码展示

4.1 章节概述

本章围绕 Spec2Vision 的关键实现展开说明，重点展示多 Agent 流水线、Agent 核心循环、工具定义、Provider 封装、配置文件管理和 AI IDE 辅助开发过程。与前一章偏重系统架构不同，本章更关注“架构如何落到代码中”，即系统如何把用户输入转化为状态对象，如何在多个 Agent 之间传递信息，如何调用生成工具，如何记录 Trace，以及如何通过配置文件控制 Mock、OpenAI、DeepSeek 等不同运行模式。

从实现角度看，本项目并没有把全部逻辑写成一个单体函数，而是将任务路由、需求澄清、规格生成、Prompt 构造、资产生成和结果评估拆分为多个模块。每个模块的职责边界较清晰，便于独立测试，也方便后续扩展新的任务类型、生成 Provider 或评估策略。

展示内容	对应实现	报告意义
多 Agent 核心循环	状态流转、条件分支、Trace 记录	展示 Agent 如何协同完成一次生成任务
工具定义与 Provider 封装	Mock、OpenAI、SVG、Evaluator	展示生成工具如何被统一调用
配置文件与安全管理	.env、settings、API Key 读取	展示不硬编码密钥与多模式切换
AI IDE 使用	Cursor 辅助编码、重构、测试	展示 AI 辅助工程开发过程

4.2 多 Agent 流水线关键实现

Spec2Vision 的主流程可以理解为一个围绕共享状态 state 展开的 Agent workflow。用户输入首先被封装为请求对象，并进入任务路由阶段。Router Agent 根据关键词、正则规则和可选 LLM 判断任务类型，并给出置信度。当置信度较低或关键信息缺失时，系统不会直接生成图片，而是进入澄清阶段，向用户提出更具体的问题；当信息足够时，系统继续进入 Requirement、Visual Spec、Prompt、Asset Manager 和 Evaluator 等后续步骤。

这种实现方式的优点在于，每个 Agent 只负责 state 的一部分字段。例如 Router Agent 写入 task_type、confidence 和 reasoning；Clarification Agent 写入 questions 和 clarification_required；VisualSpec Agent 写入结构化 visual_spec；AssetManagerAgent 写入 asset_path、provider 和 metadata；Evaluator 写入 evaluation_report。通过这种状态驱动方式，整个流程既具有连续性，又不会让某一个 Agent 承担过多职责。

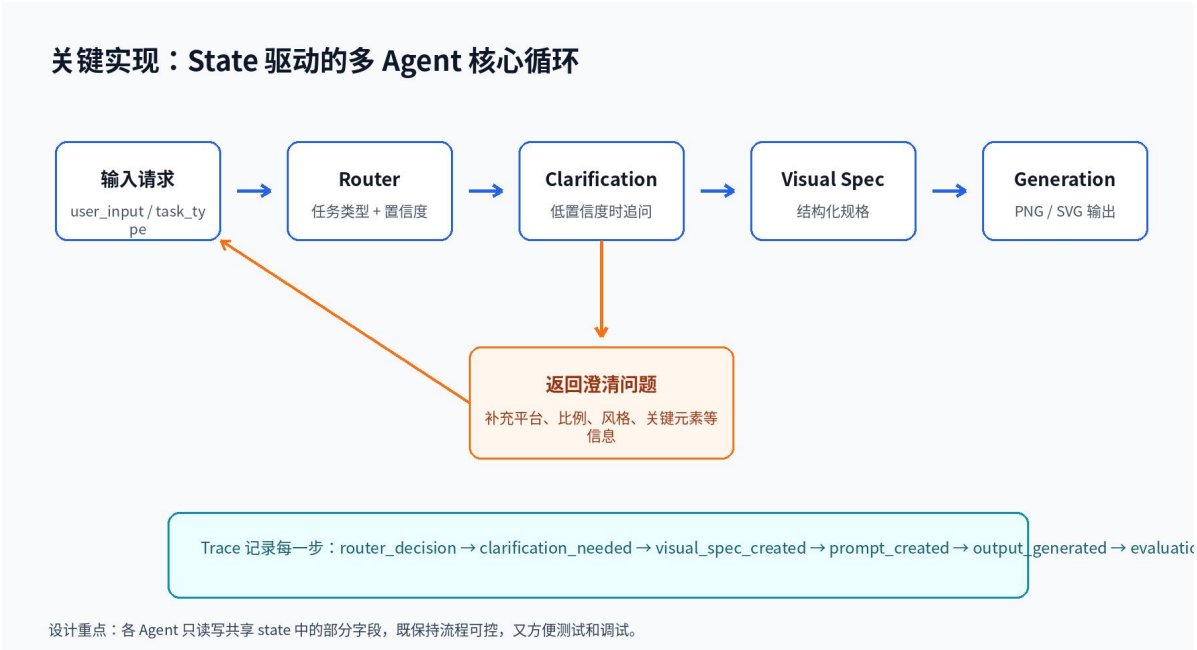


图 4-1 State 驱动的多 Agent 核心循环

Agent 核心循环展示：

```
# 代码片段 4-1：多 Agent 流水线的核心执行思想（节选 / 简化展示）
state = init_generation_state(request)

state = router_agent.run(state)
trace.add_step("router_decision", state.router_result)

if state.clarification_required and not request.skip_clarification:
    trace.add_step("clarification_needed", state.clarification_questions)
    return build_clarification_response(state)

state = requirement_agent.run(state)
state = visual_spec_agent.run(state)
state = domain_agent.run(state)
state = prompt_agent.run(state)
state = asset_manager_agent.run(state)
state = critic_agent.run(state)

trace.add_step("evaluation_completed", state.evaluation_report)
return build_generation_response(state)
```

上述代码片段体现了本项目的核心实现思想：每一步 Agent 执行后都更新 state，并将关键结果写入 Trace。这样做的好处是，系统不仅能够生成结果，还能够解释结果是怎样一步步产生的。对于课程评分中的 Agent 架构设计、工程规范性和可评估性，这种实现方式都比较直观。

4.3 Router 与 Clarification 的实现

Router Agent 是整条流水线的入口模块，主要负责判断用户输入属于哪一种视觉任务。项目支持电商主图、学术流程图、PPT 视觉封面等任务类型，不同任务会触发不同的领域增强逻辑和评估标准。**Router** 的实现并不是简单地做字符串包含判断，而是结合任务关键词、加权规则、正则模式和置信度阈值，输出 `task_type`、`confidence`、`reasoning` 和 `matched_signals` 等信息。

Clarification Agent 的作用是弥补用户 **Prompt** 中的信息缺失。很多非专业用户并不知道应该如何描述画面比例、平台尺寸、视觉风格、关键元素和文字内容。如果 **Router** 判断输入信息不足，**Clarification Agent** 会生成有针对性的澄清问题。例如，当用户只输入“帮我做一张促销图”时，系统可以追问商品类别、平台比例、促销文案和目标风格；当用户输入“帮我画一个论文方法图”时，系统可以追问论文主题、图类型、节点数量和输出格式。

Router 与澄清判断展示：

```
# 代码片段 4-2: Router / Clarification 的关键逻辑（节选 / 简化展示）
signals = collect_domain_signals(user_input)
task_type, confidence = weighted_classify(signals)

router_result = {
    "task_type": task_type,
    "confidence": confidence,
    "matched_signals": signals,
    "reasoning": explain_route(signals, task_type),
}

if confidence < ROUTER_CONFIDENCE_THRESHOLD:
    clarification_required = True
    questions = build_clarification_questions(task_type, user_input)
else:
    clarification_required = missing_required_fields(user_input, task_type)
```

这一部分实现体现了本项目对“非专业用户不会写 **Prompt**”的处理：系统不会把模糊输入直接丢给图像模型，而是先判断需求是否足够明确，再决定是否继续生成。这比传统的一次性 **Prompt** 生图方式更符合 **Agentic AI** 的交互思想。

4.4 Visual Spec 与 Prompt 构造实现

Visual Spec 是本项目最重要的中间层。它把用户自然语言需求转化为结构化字段，使后续 **Agent** 可以基于明确规格执行任务。**Visual Spec** 通常包含任务类型、标题、画面比例、关键元素、风格约束、输出格式、领域扩展字段等内容。相比直接使

用一段 Prompt，Visual Spec 更适合被代码校验、存储、传递和评估。

Prompt Agent 不直接依赖用户原始输入，而是基于 Visual Spec 生成更完整的图像生成 Prompt。这样可以减少用户表达不完整带来的问题。例如，用户只说“科技感 PPT 封面”，Visual Spec 会先补全主题、比例、风格、关键元素和约束条件，再由 Prompt Agent 转换为适合生成模型的提示词。

Visual Spec 与 Prompt 构造展示：

```
# 代码片段 4-3: Visual Spec 的结构化字段示例
visual_spec = {
    "task_type": "ppt_visual",
    "title": "智能驱动的未来架构",
    "aspect_ratio": "16:9",
    "key_elements": ["technology background", "main title", "abstract lines"],
    "style": "modern, clean, blue technology style",
    "constraints": ["clear hierarchy", "presentation friendly", "no clutter"],
    "output_format": "png"
}

prompt = prompt_agent.build_prompt(visual_spec)
trace.add_step("prompt_created", {"prompt": prompt})
```

通过 Visual Spec，中间结果不再只是不可控的自然语言描述，而是可以被测试和追踪的结构化数据。后续 Evaluator 也可以根据 Visual Spec 检查生成结果是否符合尺寸、格式、领域要求和用户约束。

关键文件与实现职责映射

层次	代表文件	核心职责	报告展示点
Agent 层	app/agents/	路由、澄清、规格生成、Prompt 构建	展示核心 Agent 逻辑
Graph 层	app/graph/visionflow_graph.py	组织 Agent 执行顺序与条件分支	展示核心循环与状态流转
Tool 层	app/tools/	图像生成、SVG、评估、文档处理	展示工具定义和 Provider 抽象
Service 层	app/services/	连接 API 请求和 Agent 流水线	展示端到端调用入口
Config 层	app/core/ + .env	集中管理 Provider、Key、Demo 模式	展示不硬编码 API Key
Test 层	tests/ + benchmark.py	验证路由、生成、评估和 API 行为	展示测试与评估证据

图 4-2 关键代码文件与实现职责映射

4.5 工具定义与 Provider 封装

在生成层，Spec2Vision 使用工具封装的方式隔离 Agent 逻辑和底层模型调用。AssetManagerAgent 不直接关心图像是由 Mock Provider、OpenAI Images API 还是本地 SVG Diagram Generator 生成，而是通过统一接口选择合适工具。这种设计使系统可以在无 API Key 的情况下使用 Mock Provider 进行测试，也可以在真实场景下切换到 OpenAI Images API 生成高质量图片。

本项目还实现了 Evaluator 工具，用于对生成结果进行格式检查、尺寸检查、图像启发式指标分析和 Rubric 评分。生成工具负责“产出资产”，评估工具负责“判断质量”，二者共同构成完整的生成闭环。

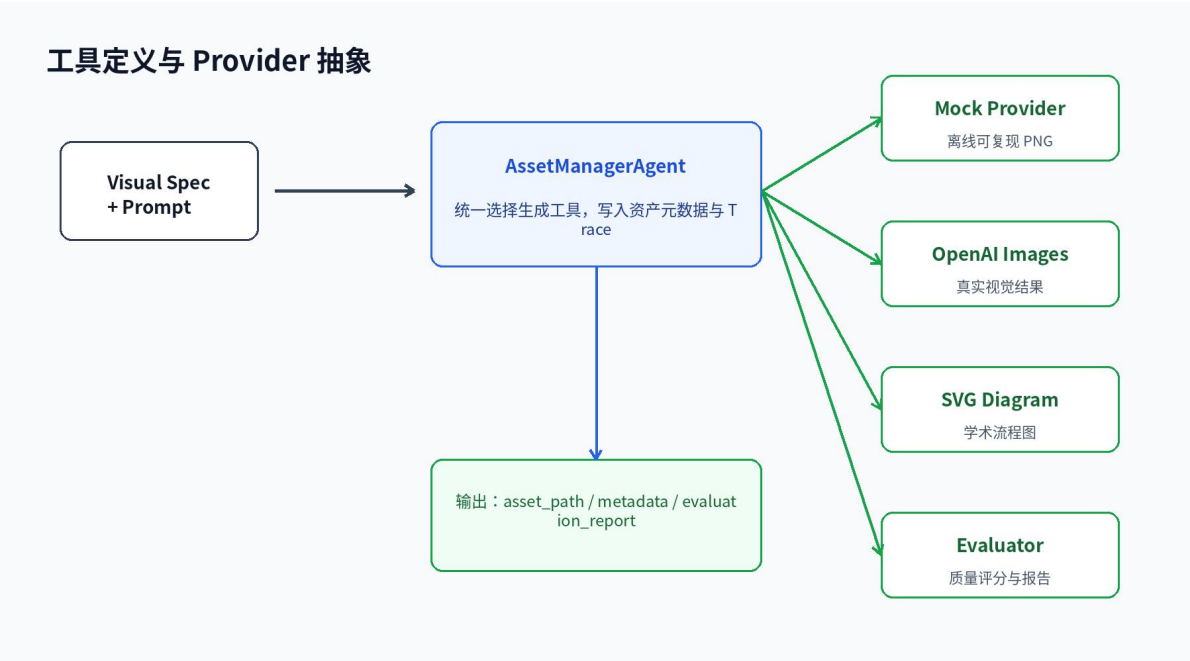


图 4-3 工具定义与 Provider 抽象关系

工具名称	主要职责	使用场景
create_visual_spec	根据用户需求生成结构化 Visual Spec	MCP / Agent 调用
generate_visual_asset	根据 Visual Spec 和 Prompt 生成 PNG 或 SVG	资资产生成
evaluate_visual_asset	对生成结果进行确定性检查、启发式评估和 Rubric 评分	质量评估
query_generation_trace	查询任务执行过程中的 Agent Trace	可观测性

Provider 选择逻辑展示：

```
# 代码片段 4-4: Provider 选择与工具封装思想（节选 / 简化展示）
if settings.IMAGE_PROVIDER == "mock":
```

```
generator = MockImageGenerator()
elif settings.IMAGE_PROVIDER == "openai":
    generator = OpenAIImageGenerator(api_key=settings.OPENAI_API_KEY)
else:
    raise ValueError("Unsupported image provider")

asset = generator.generate(prompt=prompt, aspect_ratio=visual_spec.aspect_ratio)
evaluation = evaluator.evaluate(asset_path=asset.path, visual_spec=visual_spec)
```

4.6 配置文件与安全管理

为了避免在代码中硬编码 API Key，本项目使用 .env 和配置模块统一管理运行参数。开发者可以通过 IMAGE_PROVIDER、LLM_PROVIDER、DEMO_MODE、OPENAI_API_KEY、DEEPSEEK_API_KEY 等环境变量控制系统运行模式。默认情况下，系统可以使用 Mock Provider 离线运行，方便测试和课程验收；当需要真实模型能力时，再通过 .env 配置相应的 API Key。

这种配置方式有两个优点。第一，开发、测试、演示和部署可以使用不同配置，不需要修改业务代码。第二，API Key 不会出现在 Git 仓库中，降低敏感信息泄露风险。项目中的 .env.example 只提供变量名和默认模式，真实密钥由本地 .env 提供。

.env 配置展示：

```
# 代码片段 4-5：环境变量配置示例
IMAGE_PROVIDER=mock
LLM_PROVIDER=mock
DEMO_MODE=false
VISION_EVALUATOR_PROVIDER=none
OCR_PROVIDER=none

# 使用真实模型时再配置：
# LLM_PROVIDER=deepseek
# DEEPSEEK_API_KEY=your_deepseek_api_key
# IMAGE_PROVIDER=openai
# OPENAI_API_KEY=your_openai_api_key
```

配置项	作用	设计说明
IMAGE_PROVIDER	控制图像生成方式，支持 mock / openai	默认 mock，便于离线测试
LLM_PROVIDER	控制文本模型，支持 mock / deepseek / openai	默认 mock，避免强依赖外部服务
DEEPSEEK_API_KEY	DeepSeek API 密钥	只放在本地 .env，不提交仓库
OPENAI_API_KEY	OpenAI API 密钥	只在 openai 模式下需要
DEMO_MODE	演示模式开关	可强制走稳定演示路径

4.7 AI IDE 使用过程展示

本项目在开发过程中使用 Cursor 作为 AI IDE 辅助工具。图 4-4 展示了项目开发阶段的实际工作界面：中间区域运行本地 VisionFlow 前端，左侧显示 Agent 交互与终端反馈，右侧展示 Cursor 对 Git 提交、测试结果和代码改动的辅助分析。通过这种开发方式，需求实现、错误定位、测试补全和文档整理可以在同一工作空间中协同完成。

在实际开发中，Cursor 主要用于辅助生成 Agent 类和服务层函数骨架、解释 pytest 报错、梳理 Provider 封装边界、检查 README 与规格文档的一致性。项目的关键逻辑仍以系统需求、架构约束和测试结果为依据，并经过本地运行、Git 提交和回归测试验证。

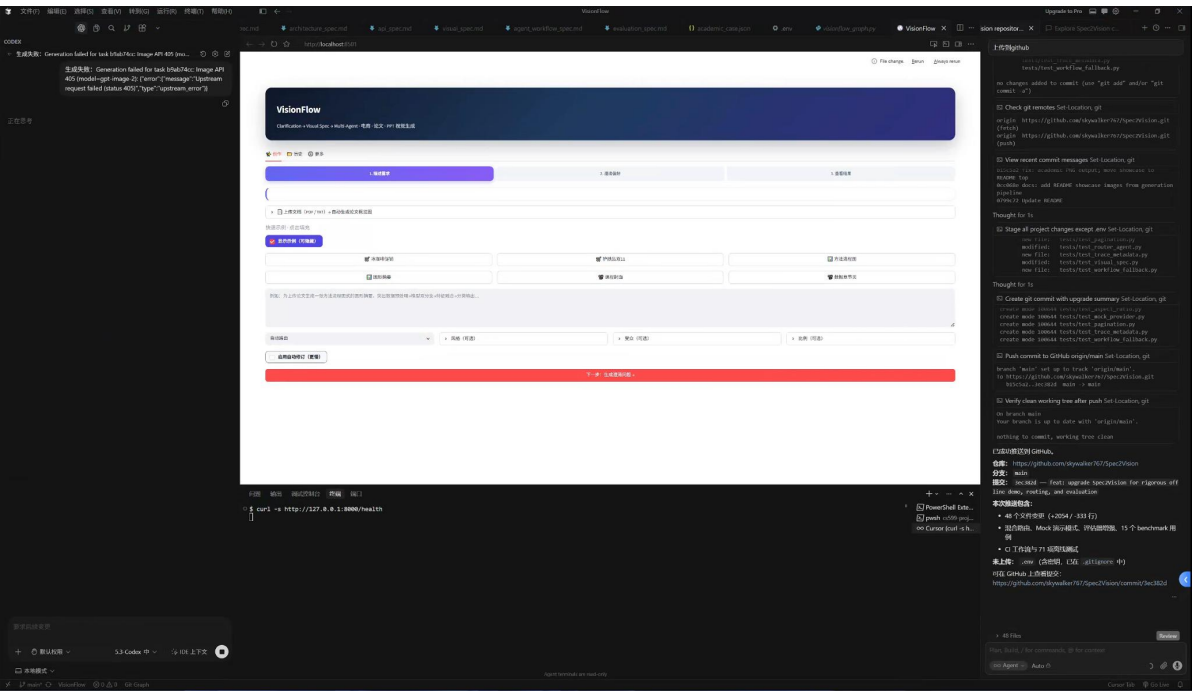


图 4-4 Cursor 辅助开发与调试界面

图 4-4 体现了 Cursor 在本项目中的辅助开发过程。该界面同时展示了本地 Demo 运行状态、终端命令、错误信息、Git 变更摘要和提交记录，说明项目开发并非停留在文档设计阶段，而是经过了实际编码、调试、运行和版本管理。

4.8 本章小结

本章从代码实现角度说明了 Spec2Vision 的关键模块。项目的核心不是简单调用图像生成 API，而是围绕多 Agent 流水线构建了一套可控、可追踪、可评估的视觉

生成流程。Router 和 Clarification 解决用户 Prompt 模糊和信息不完整的问题；Visual Spec 和 Prompt Agent 将自然语言需求转化为可执行规格；AssetManagerAgent 和 Provider 封装实现多种生成方式切换；Evaluator 和 Trace 使生成结果能够被解释和评估；配置文件则保证系统可以在本地测试、演示和真实调用之间灵活切换。

通过这些关键实现，Spec2Vision 在工程上形成了从用户输入、Agent 决策、工具调用、资产生成到结果评估的完整闭环，也为后续扩展新的视觉任务类型、接入新的生成模型和增强评估能力提供了清晰基础。

五、测试与评估

5.1 测试与评估目标

本章主要说明 Spec2Vision 在功能正确性、Agent 行为稳定性、Benchmark 指标、Demo 展示效果以及结果可解释性方面的测试与评估情况。对于一个多 Agent 视觉内容生成系统而言，测试目标不仅是验证接口是否能够正常返回结果，还需要验证系统是否能够正确理解用户需求、在信息不足时触发澄清、生成可执行的 Visual Spec，并最终输出可下载、可评估、可追踪的视觉资产。

因此，本项目将测试与评估分为四个层次：第一层是功能测试，验证 API、UI、文件处理、任务管理和 Provider 调用是否正常；第二层是 Agent 行为评估，重点检查 Router、Clarification、VisualSpec、Prompt 和 Evaluator 等 Agent 的输出是否符合预期；第三层是 Benchmark 测试，通过固定用例集批量验证路由准确率、生成成功率和评估稳定性；第四层是 Demo 证据，包括生成结果截图、Trace 时间线、evaluation.json 和录屏材料，用于证明系统端到端流程完整可复现。

测试与评估整体框架

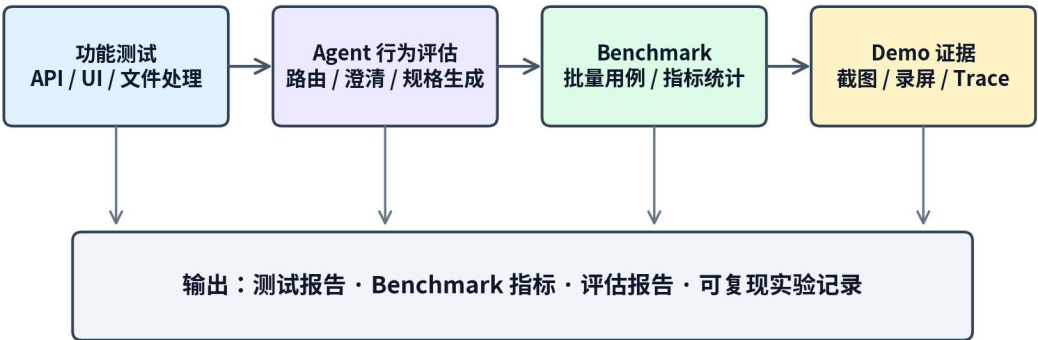


图 5-1 测试与评估整体框架

5.2 功能测试

功能测试主要用于验证系统各模块是否能够按照预期完成基本任务。本项目使用 `pytest` 作为主要测试框架，并将 `Mock Provider` 作为默认测试模式。`Mock` 模式不依赖外部 `API Key`，可以在本地环境和 `CI` 环境中稳定运行，避免由于网络、额度或模型返回不稳定导致测试结果不可复现。

功能测试覆盖了后端接口、任务生成流程、文件处理、配置读取、数据库任务记录、资产下载和错误路径等内容。后端接口测试重点验证 `/generate`、`/clarify`、`/extract`、`/tasks` 等 `API` 的请求和响应结构；文件处理测试重点验证文本 `PDF`、空文件和不支持格式的处理逻辑；任务管理测试验证任务创建、查询、下载和删除流程；错误路径测试则用于确认系统在缺少 `API Key`、`Provider` 配置错误、输入非法等情况下能够返回明确错误信息。

测试类别	测试内容	评估重点
API 测试	<code>/generate</code> 、 <code>/clarify</code> 、 <code>/extract</code> 、 <code>/tasks</code> 等接口	请求参数、响应结构、错误处理
文件处理测试	<code>PDF / TXT</code> 上传与解析	文本抽取、空文件、扫描件提示、不支持格式处理
任务管理测试	任务创建、查询、资产下载、删除	任务状态、文件路径、数据库记录一致性
配置测试	<code>.env</code> 、 <code>Provider</code> 、 <code>API Key</code> 配置	默认 <code>Mock</code> 模式、错误配置提示、密钥不硬编码
端到端测试	从输入需求到生成资产和评估报告	完整流水线可运行、输出工件完整

表 5-1 功能测试覆盖范围

从测试结果来看，项目在默认 Mock 模式下可以完成端到端生成流程，测试报告显示 147 passed、1 skipped。由于 Mock Provider 的生成结果具有确定性，因此非常适合用于课程验收、回归测试和 CI 自动化检查。真实 OpenAI Images API 模式则主要用于展示更接近实际视觉效果的生成结果。

5.3 Agent 行为评估

Agent 行为评估是本项目区别于普通 Web 应用测试的重点。Spec2Vision 的核心不是单一接口调用，而是由多个 Agent 协作完成任务。因此，除了验证最终是否生成图片，还需要检查每个 Agent 的中间输出是否合理。

Router Agent 的评估重点是任务分类是否正确。系统需要将用户输入识别为 ecommerce_banner、academic_figure 或 ppt_visual 等任务类型，并输出 confidence、reasoning、matched_signals 和 clarification_required 等字段。对于输入信息明显不足的情况，Router 不应该盲目进入生成阶段，而应触发澄清。

Clarification Agent 的评估重点是澄清问题是否针对缺失信息。例如，当用户只输入“做一张促销海报”时，系统应该追问商品类型、平台比例、促销文案和视觉风格；当用户输入“画论文方法图”时，系统应该关注图类型、研究主题、节点关系和输出比例。

VisualSpec Agent 和 Prompt Agent 的评估重点是结构化规格与最终 Prompt 是否保持一致。Visual Spec 应该包含任务类型、标题、关键元素、尺寸比例、风格约束和领域扩展字段；Prompt 则应该由 Visual Spec 派生，而不是重新随机生成。这样可以保证用户需求在 Agent 之间传递时不丢失。

Agent	行为评估重点	通过标准
TaskRouterAgent	任务类型识别、置信度、是否需要澄清	分类结果与用例预期一致，低置信输入触发澄清
ClarificationAgent	是否能发现缺失信息并提出问题	问题与任务类型相关，能够补全关键约束
VisualSpecAgent	是否生成结构化规格	字段完整、约束明确、与用户需求一致
PromptAgent	是否根据 Visual Spec 构造 Prompt	Prompt 保留关键元素和约束，不偏离规格
AssetManagerAgent	是否正确选择 Provider 并保存资产	输出 PNG / SVG 文件，路径和元数据完整
Evaluator /	是否输出可解释评估报告	包含分项评分、证据、警告和建议

CriticAgent		
-------------	--	--

表 5-2 Agent 行为评估维度

5.4 结果评估机制

生成结果评估采用分层 Evaluator 设计。第一层是确定性检查，主要验证文件格式、尺寸比例、SVG 结构、输出路径等硬性要求。这一层不依赖模型判断，能够稳定发现明显错误。第二层是启发式图像指标，例如图像 entropy、对比度、边缘密度和空白区域检测，用于识别过于单调、空白或信息量不足的图像。第三层是可选 VLM 评估，在配置视觉模型 API Key 后，可以进一步进行语义和美学层面的判断。

最终 Evaluator 会将不同层次的结果整理成 Rubric 评分。Rubric 包括 visual_validity、spec_completeness、requirement_alignment、domain_fit、traceability 和 reproducibility 等维度。这样可以让评估结果不仅是一个总分，还包含每个维度的理由、证据和改进建议。对于课程项目而言，这种评估方式可以增强结果可信度，也方便老师检查系统是否真正具备可解释性。

分层 Evaluator 评估流程

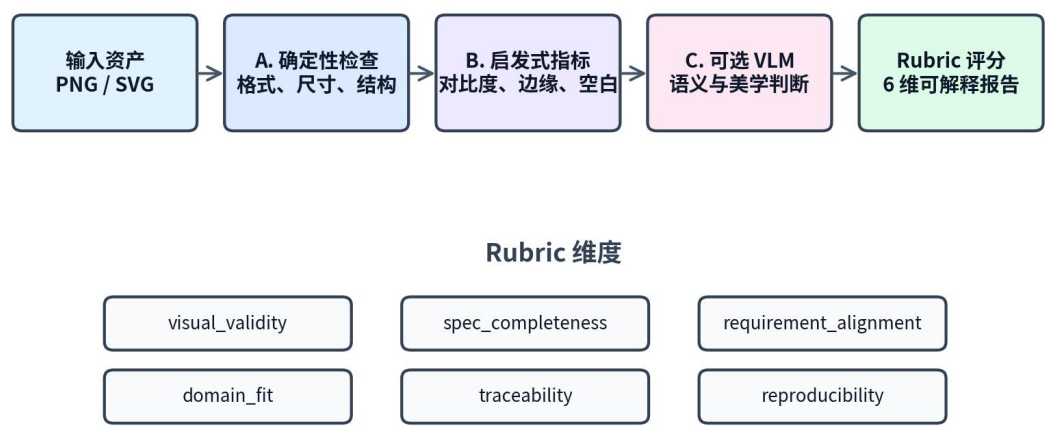


图 5-2 分层 Evaluator 评估流程

5.5 Benchmark 测试

Benchmark 测试用于验证系统在一组固定用例上的整体表现。项目中的

benchmark 用例覆盖电商主图、学术流程图和 PPT 视觉设计三类任务，每个用例包含用户输入、期望任务类型、生成参数和评估目标。通过批量运行 benchmark，可以观察路由准确率、生成成功率、平均评估分数和失败用例原因。

本项目将 routing_accuracy 作为关键指标之一，测试阈值设置为不低于 0.75。这是因为视觉生成任务的前置条件是正确识别任务类型，如果路由错误，后续澄清问题、Visual Spec、Prompt 和评估标准都可能偏离。generation_success_rate 则用于衡量系统是否能够稳定输出资产文件。由于 Mock Provider 不依赖外部服务，在本地 benchmark 中生成成功率应保持稳定。

```
python benchmark.py --benchmark
# 或
make benchmark
```

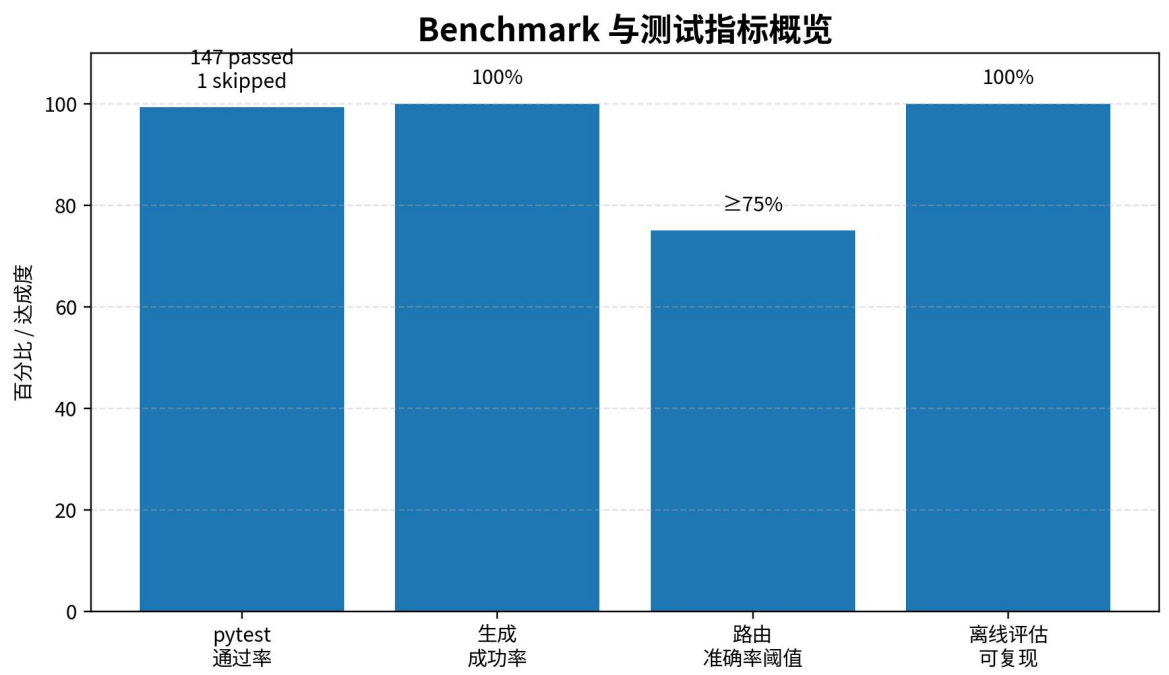


图 5-3 Benchmark 与测试指标概览

指标	含义	项目目标
routing_accuracy	路由结果与 benchmark 预期任务类型的一致比例	不低于 0.75
generation_success_rate	成功生成资产文件的用例比例	Mock 模式下保持稳定
evaluator_avg_score	Evaluator 输出的平均评估分数	用于观察不同任务质量趋势
trace_completeness	Trace 是否包含关键 pipeline_step	覆盖路由、规格、生成、评估阶段

表 5-3 Benchmark 核心指标

5.6 Demo 展示与系统运行证据

Demo 展示选取电商主图生成场景，覆盖从自然语言需求输入、自动路由、偏好澄清到生成准备的关键环节。图 5-4 展示了 VisionFlow 主界面中的需求输入阶段，用户可以选择快速示例，也可以直接输入完整 Prompt；系统会根据输入内容进行任务识别，并进入后续澄清或生成流程。

在偏好澄清阶段，系统已经识别出当前任务属于电商主图场景，并围绕目标投放平台、营销目标、整体视觉风格、图片比例、促销强度、商品占比、价格标签位置和冰块呈现方式等字段进行补全。通过该交互，模糊 Prompt 被进一步转化为可执行的 Visual Spec 输入条件。

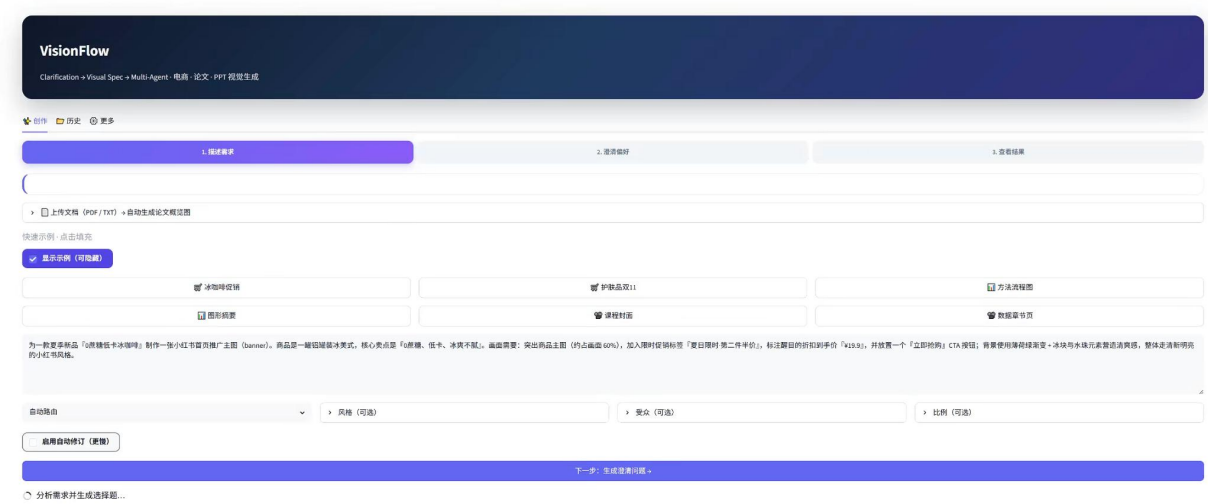


图 5-4 Demo 需求输入与任务路由界面

表 5-4 Demo 展示证据与对应能力

Demo 证据	实际展示内容	说明
需求输入与路由	VisionFlow 主界面、快速示例、Prompt 输入区和自动路由入口	证明系统能够从自然语言需求进入端到端生成流程
偏好澄清	目标平台、营销目标、视觉风格、比例、促销强度、商品占比等澄清项	证明系统能够将模糊 Prompt 补全为可执行生成条件
Visual Spec / 生成准备	任务类型、比例、风格、关键元素和约束字段被结构化整理	证明生成依据来自结构化规格，而不是一次性 Prompt
结果评估与追踪	Evaluation Report、Rubric 评分和 Agent Trace 记录	证明生成结果可解释，执行过程可追踪
开发与调试证据	Cursor 中的前端运行、终端命令、错误信息、Git 提交与测试反馈	证明项目经过实际工程开发、调试和版本管理

5.7 测试结论与不足

综合功能测试、Agent 行为评估、Benchmark 和 Demo 展示结果,Spec2Vision 已经完成从用户需求输入到视觉资生产成、评估和追踪的完整闭环。测试结果说明系统在默认 Mock 模式下具有较好的可复现性,可以稳定完成课程验收所需的核心流程;在 OpenAI Images API 模式下,系统能够生成更接近真实使用场景的视觉结果。

同时,当前测试仍存在一些可以继续改进的方向。首先,离线 Mock 生成图像主要用于验证流程正确性,视觉质量不能完全代表真实模型效果。其次,启发式评估指标能够发现空白、尺寸错误和低信息量图像,但对美学和语义准确性的判断仍然有限,需要结合 VLM 或人工评价进一步增强。第三,Benchmark 用例数量还可以继续扩展,尤其是增加更多边界输入、跨语言输入和复杂约束输入,以提高系统评估的全面性。

总体来看,本项目的测试与评估设计不是只关注“程序能否运行”,而是从功能正确性、Agent 行为合理性、结果质量、过程可追踪性和 Demo 可验证性多个角度进行检查。这种测试思路与 Agentic AI 系统的特点相匹配,也为后续系统升级和扩展提供了依据。

六、系统升级与扩展

6.1 升级目标与设计原则

在前面章节中,Spec2Vision 已经完成从用户模糊需求到视觉资生产成的核心闭环,包括任务路由、需求澄清、Visual Spec 生成、Prompt 构造、资生产成、质量评

估和 Agent Trace 记录。第六章关注的是系统后续如何继续升级与扩展，使其从课程项目进一步演进为更加通用、稳定、可复用的 Agentic AI 视觉生成平台。

本项目的升级目标不是简单增加更多模型接口，而是围绕三个方向展开：第一，扩展任务边界，让系统支持更多视觉内容类型；第二，增强工程能力，让系统在部署、安全、监控和成本控制方面更加完善；第三，提升 AI 能力，让系统从“单次生成”逐步升级为“自动澄清、自动评估、自动修订”的闭环智能体。

在设计原则上，系统后续升级仍然坚持 Spec-first 和 Agent modularity 两个核心思路。Spec-first 指所有生成任务都先转化为结构化规格，避免直接依赖模糊 Prompt。Agent modularity 指不同能力通过独立 Agent 或 Tool 接入，而不是不断堆叠在一个大函数或单个模型调用中。这样可以保持系统结构清晰，也便于新增任务域、替换模型和扩展评估标准。

6.2 可扩展架构设计

Spec2Vision 当前已经将核心流程拆分为多个相对独立的模块，包括 UI 层、API 层、Agent 编排层、工具与 Provider 层、评估层和存储层。后续扩展时，可以继续沿用这一分层架构，在不破坏主流程的前提下增加新的领域 Agent、新的生成 Provider、新的知识库和新的评估策略。

在 Agent 层，系统可以为不同视觉任务增加独立的 Domain Agent。例如，当前重点覆盖电商主图、学术流程图和 PPT 视觉封面，后续可以扩展到社交媒体配图、品牌海报、论文 graphical abstract、教学课件插图、短视频封面等场景。每个新增领域只需要定义对应的 Visual Spec 字段、Prompt 模板和评估规则，就可以接入现有生成流水线。

在工具与 Provider 层，系统可以保持统一的 Provider 接口，将 OpenAI Images API、Mock Provider、本地模型或未来的第三方图像模型封装为可插拔组件。这样做的优势是上层 Agent 不需要关心底层模型细节，只需要提交结构化规格和生成请求。未来如果需要切换模型、增加本地推理或进行成本优化，也可以在 Provider 层完成。

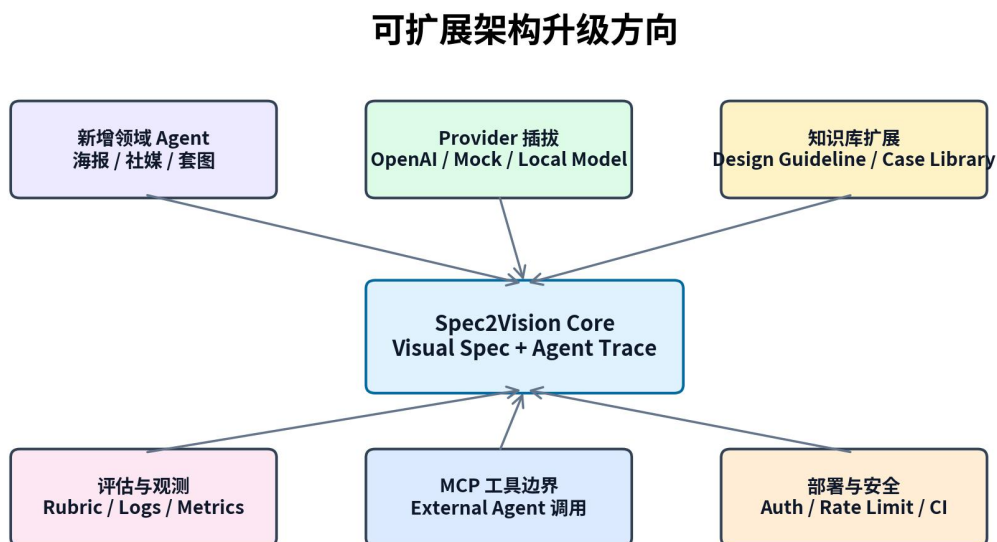


图 6-1 Spec2Vision 可扩展架构升级方向

扩展层级	扩展方向	预期价值
领域 Agent	新增社媒配图、品牌海报、课程插图、论文图形摘要等任务域	扩大系统适用范围，提升项目应用价值
Provider 层	接入本地模型、第三方图像模型和多模型路由策略	降低模型依赖，支持成本与效果平衡
知识库层	补充设计规范、平台尺寸规范、优秀案例和课程模板	增强 Agentic RAG 的领域指导能力
评估层	引入 VLM 评估、A/B 对比和自动修订建议	提升结果质量判断的可信度
部署层	增加鉴权、限流、日志、监控和队列任务	提升系统稳定性和工程完整度

6.3 功能扩展方向

从产品能力角度看，Spec2Vision 后续可以优先扩展三个方向。第一是多场景视觉生成，即在原有电商、学术和 PPT 三类任务基础上，增加更多面向真实学习和办公场景的视觉任务。第二是多版本生成与对比，即同一个 Visual Spec 可以生成多个候选结果，并通过 Evaluator 或用户偏好选择更合适的版本。第三是批量生成能力，即允许用户一次输入多个需求或上传一个文档，系统自动拆分为多个视觉生成任务。

多版本生成和自动对比对于视觉任务尤其重要。因为图像生成本身具有一定随机性，同一规格可能产生不同构图和风格。如果系统能够自动生成多个版本，并从尺寸合规、主体完整、文字可读性、领域匹配度等维度进行比较，就可以减少用户反复手

动尝试的成本。

批量生成能力则更适合课程汇报、论文展示和商业宣传场景。例如，用户可以上传一份课程项目说明，系统自动识别其中需要展示的流程圖、架构图和封面圖，并分别生成对应 Visual Spec。这样可以让 Spec2Vision 从单张图片生成工具扩展为面向项目展示的视觉资产生成助手。

功能方向	具体升级内容	关联模块
多场景生成	支持社媒配图、品牌海报、课程插图、短视频封面等	Domain Agent / Visual Spec
多版本对比	同一需求生成多个候选结果，并自动评估排序	Asset Manager / Evaluator
批量生成	根据文档或多个 Prompt 批量生成视觉资产	Document Extractor / Pipeline
交互式修改	用户对结果提出修改意见，系统转化为新的 Spec delta	Clarification Agent / Prompt Agent
模板复用	保存常用 Visual Spec 模板，用于类似任务快速生成	Storage / RAG Knowledge Base

6.4 AI 能力演进路径

AI 能力的演进是系统升级的核心。当前系统已经具备“需求理解—规格生成—资产生成—结果评估”的基本闭环，下一阶段可以进一步增强主动澄清、自动修订和长期知识积累能力。

在主动澄清方面，系统可以从固定模板问题升级为上下文感知式问题生成。也就是说，Clarification Agent 不只根据任务类型提问，还可以结合用户输入中的具体词语、领域知识和历史生成结果，提出更精准的问题。例如，当用户输入“论文方法图”时，系统可以判断是否需要追问模型结构、数据处理流程、实验阶段数量和图示风格；当用户输入“电商主图”时，系统可以追问商品类别、促销文案、平台比例和目标人群。

在自动修订方面，系统可以将 Evaluator 的结果反馈给 Prompt Agent 或 VisualSpec Agent，形成生成—评估—修订的闭环。如果评估结果发现图片比例不符合要求、主体元素缺失或文字层级不清晰，系统可以自动生成修改建议，并重新构造 Prompt 进行二次生成。这样 Spec2Vision 就可以从一次性生成系统升级为具备自我改进能力的 Agent。

在知识积累方面，系统可以将成功案例、用户选择结果和评分结果沉淀到本地知识库中。未来 Agentic RAG 不仅可以检索静态设计规范，还可以检索历史案例和项

目模板，从而为新任务提供更准确的生成参考。

AI 能力演进路径：从一次生成到闭环优化

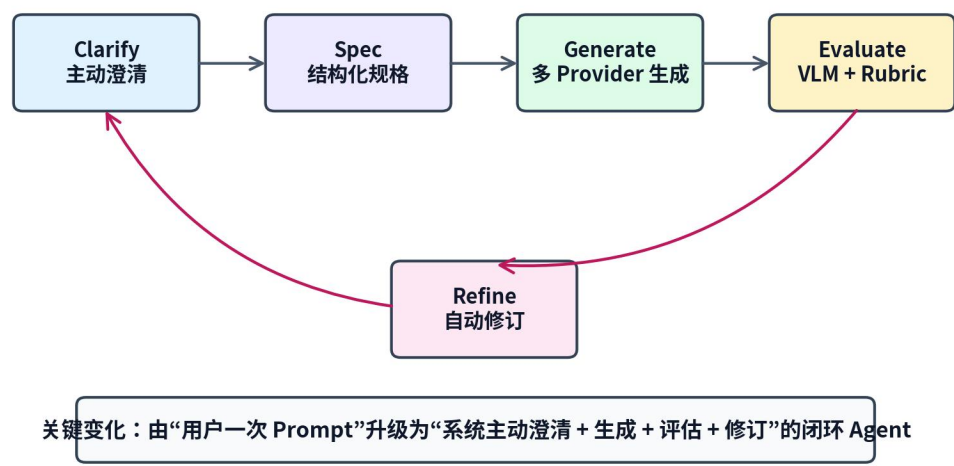


图 6-2 Spec2Vision AI 能力从一次生成到闭环优化的演进路径

6.5 工程化升级方向

除了 AI 能力本身，工程化能力也是系统后续升级的重要方向。一个可持续演进的 Agentic AI 应用需要关注稳定性、安全性、可观测性和成本控制。Spec2Vision 后续可以围绕部署、鉴权、限流、日志、监控、任务队列和缓存机制继续增强。

在部署方面，系统可以将 FastAPI 后端、Streamlit 前端和任务存储服务通过 Docker Compose 或云服务器统一部署，并提供可访问的 Demo URL。在安全方面，可以增加用户鉴权、API Key 隔离、文件上传限制和请求频率限制，避免模型接口被滥用。在可观测性方面，可以对每个 Agent 步骤记录耗时、输入输出摘要、错误类型和 Provider 调用结果，并形成可查看的日志面板。

在成本控制方面，系统可以根据任务复杂度选择合适的模型和生成策略。例如，简单测试场景使用 Mock Provider，正式视觉生成使用 OpenAI Images API，文本规格生成可以优先使用 DeepSeek API。对于重复需求，可以通过缓存 Visual Spec、Prompt 和生成结果减少重复调用。

工程方向	升级内容	作用
稳定性	任务队列、重试机制、超时控制、错误分类	减少长任务失败对用户体验的影响
安全性	鉴权、限流、文件类型检查、API Key 隔	保护系统接口和外部模型资

	离	源
可观测性	Agent Trace、日志面板、耗时统计、错误统计	便于定位问题和优化性能
成本控制	Provider 路由、缓存、Mock 测试模式、批量请求优化	降低真实模型调用成本
交付能力	Docker 部署、CI/CD、在线 Demo、报告自动导出	提升项目展示和持续迭代效率

6.6 下一阶段计划

结合课程项目进度和系统当前状态，下一阶段计划可以分为短期、中期和长期三个阶段。短期目标主要是完善最终展示材料，包括云端部署、Demo 录屏、README、测试报告和课程报告。中期目标主要是增强系统能力，包括扩展任务域、强化 RAG 知识库、完善自动评估与修订闭环。长期目标则是将 Spec2Vision 扩展为一个可复用的视觉生成平台，支持团队协作、模板管理、多模型协同和持续优化。

短期阶段强调可展示性和可复现性。系统需要保证评审者能够通过 README、Demo 用例、测试报告和在线服务快速理解项目价值。中期阶段强调能力增强，重点是让系统从“能生成”变成“能更稳定地生成”。长期阶段强调平台化，使系统不仅服务于单次课程项目，也可以迁移到更多学习、科研和办公场景中。

下一阶段路线图

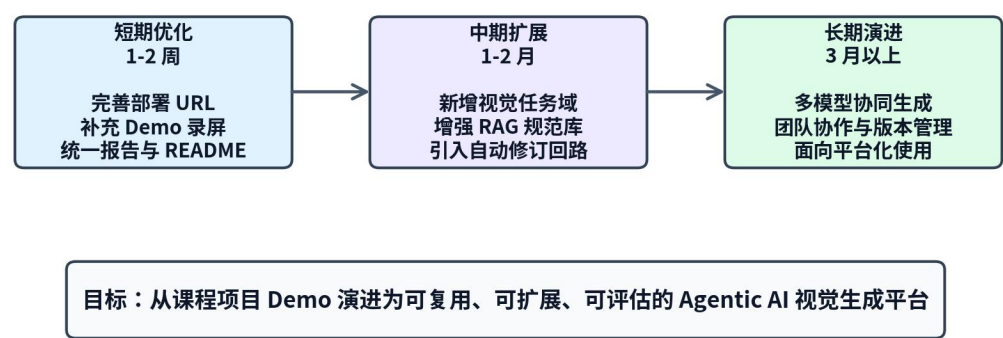


图 6-3 Spec2Vision 下一阶段升级路线图

阶段	重点任务	交付结果
短期	完善在线 Demo、答辩材料、测试报告和	可访问、可演示、可复现

	README 展示效果	
中期	新增领域 Agent, 扩展 RAG 规范库, 引入自动修订闭环	更强的任务覆盖和生成稳定性
长期	支持团队协作、模板市场、多模型调度和平台化部署	可复用的 Agentic AI 视觉生成平台

6.7 本章小结

本章围绕系统升级与扩展进行了分析。Spec2Vision 后续的发展重点可以概括为三个方面：在产品能力上扩展更多视觉任务域，在 AI 能力上形成自动澄清、自动评估和自动修订闭环，在工程能力上增强部署、安全、可观测性和成本控制。

从架构角度看，当前系统采用 Visual Spec 中间层、多 Agent 分工、Provider 插拔和 Trace 记录的设计，为后续扩展提供了较好的基础。新增领域 Agent、替换模型 Provider、扩展知识库或增强 Evaluator 时，都可以在现有架构中以较低成本接入。

总体而言，Spec2Vision 的下一阶段目标不是简单增加功能数量，而是让系统在“可扩展、可解释、可评估、可部署”四个方面持续提升。通过这些升级，项目可以从课程级视觉生成 Demo 进一步演进为具有实际应用潜力的 Agentic AI 原生视觉内容生成系统。

6.8 与评分标准的对应关系

从评分标准来看，第六章主要对应“升级扩展设想”部分，重点体现系统未来演进的前瞻性、可行性和设计深度。本项目的扩展设想不是停留在概念层面，而是围绕当前已有架构继续展开，因此后续计划能够直接映射到现有代码结构、Agent 设计、Provider 封装和评估模块中。

评分维度	本章对应内容	体现方式
前瞻性	自动澄清、自动评估、自动修订、多模型协同	体现 Agentic AI 能力从单次生成走向闭环优化
可行性	新增 Domain Agent、Provider 插拔、RAG 知识库扩展	基于当前架构继续扩展，不需要推翻已有设计
深度	同时考虑产品、AI、工程化和部署能力	从功能扩展、AI 演进、稳定性和成本控制多个层面展开
加分项关联	MCP、Agentic RAG、部署、安全、可观测性	与课程加分方向保持一致，便于后续答辩展示

因此，第六章的核心作用是说明 Spec2Vision 具有继续演进的空间和清晰路径。它不仅展示了未来可以增加哪些功能，也说明了为什么这些功能能够在现有系统中实现，以及这些升级如何进一步提升项目的工程价值和实际应用价值。

七、课程总结

7.1 项目回顾

本课程项目以 Spec2Vision 为主题，围绕“从模糊需求到可下载视觉资产”的目标，完成了一个基于 Visual Spec 的多 Agent 视觉内容生成系统。项目最初的设想只是希望让用户输入一句自然语言需求后生成图片，但在实际设计和实现过程中，我逐渐意识到，真正困难的并不是单纯调用图像生成模型，而是如何让系统正确理解用户需求、如何处理模糊 Prompt、如何保证生成过程可追踪、以及如何对输出结果进行可信评估。

因此，项目最终没有采用简单的“Prompt 输入框 + 图片输出”模式，而是设计了包含 Router、Clarification、Requirement、Visual Spec、Domain Agent、Prompt、Asset Manager 和 Evaluator 在内的多 Agent 流水线。系统首先识别任务类型，在信息不足时主动澄清，然后生成结构化 Visual Spec，再调用 Mock Provider、OpenAI Images API 或 SVG Diagram Generator 生成视觉资产，最后通过 Evaluator 输出评估报告，并通过 Agent Trace 记录完整过程。

从结果来看，Spec2Vision 已经覆盖了课程要求中的多个核心能力点，包括 SDD 规格文档、Agentic AI 原生开发、多 Agent 工作流、LangGraph 编排、MCP Tool Server、Agentic RAG、FastAPI 后端、Streamlit 前端、Docker 部署、pytest 测试和 CI 集成。整个项目从产品定义、架构设计、关键实现到测试评估，形成了比较完整的工程闭环。

7.2 个人收获

通过本项目，我最大的收获是对 Agentic AI 应用开发有了更加完整的理解。课程开始前，我对 AI 应用的理解更多停留在“调用大模型接口完成一个功能”这一层面，关注点主要是 Prompt 怎么写、模型返回结果是否可用。而完成 Spec2Vision 后，我认识到一个真正可用的 Agentic AI 系统不仅需要模型能力，还需要清晰的问题定义、结构化规格、模块化架构、可观测执行过程和可靠的评估机制。

在需求分析方面，我学习到不能直接把用户输入当成完整需求。很多用户并不是专业设计师，也不熟悉 Prompt 的写法，他们可能只会用模糊语言表达目标，例如“高级一点”“科技感”“适合汇报”。如果系统直接生成结果，就会把需求理解的压力全部交给用户。因此，我在项目中设计了 Clarification Agent，让系统能够主动发现模糊点并提出澄清问题。这让我意识到，AI 系统不应该只是被动响应，而应该在必要时主动帮助用户完善任务。

在系统设计方面，我对 SDD 方法有了更具体的体会。Product Spec 帮助我明确系统要解决什么问题、面向什么用户、支持哪些功能；Architecture Spec 帮助我拆分模块、明确 Agent 职责和数据流；API Spec 帮助我在实现前先定义接口契约。相比边写代码边调整的方式，先写规格文档可以显著减少后期返工，也能让项目结构更加清晰。

个人学习与能力成长路径

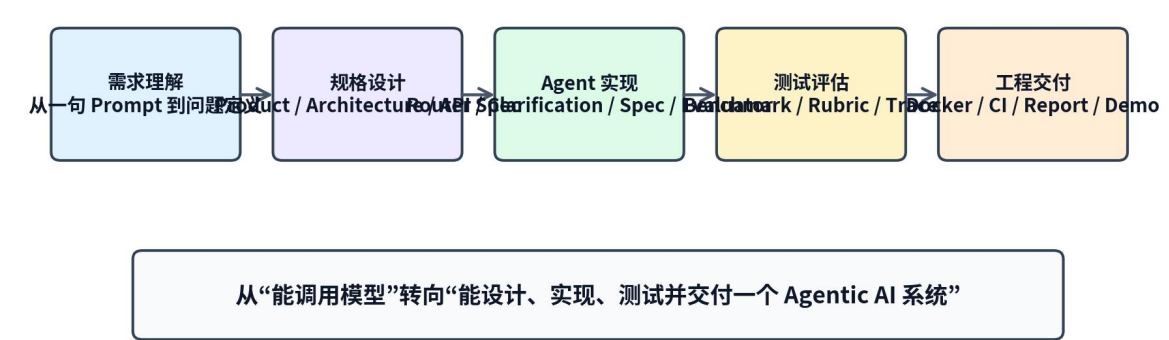


图 7-1 个人学习与能力成长路径

7.3 工程思维转变

本项目也让我在工程思维上发生了比较明显的转变。项目初期，我更关注功能是否能够跑通，例如能不能调用模型、能不能生成图片、界面是否能展示结果。但随着系统复杂度增加，我逐渐开始关注功能背后的工程质量：需求是否被正确理解，模块之间是否解耦，数据结构是否稳定，错误是否能够被处理，结果是否能够被复现，测试是否能够覆盖关键路径。

这种转变在 Agent 设计中体现得很明显。如果把所有逻辑都放在一个大模型调用中，短期看起来实现简单，但系统很难调试，也很难解释结果。后来我将任务拆分为多个 Agent，每个 Agent 只负责一个明确步骤，例如 Router 负责分类，Clarification 负责追问，VisualSpec Agent 负责规格化，Evaluator 负责评分。这样虽然代码结构更复杂，但每个模块更容易测试，也更方便后续扩展。

同时，我也认识到测试和评估不是项目最后的附属部分，而是系统设计的一部分。Spec2Vision 中的 Benchmark、Rubric、Trace 和 pytest 测试共同构成了质量保障机制。通过这些机制，我可以判断系统是否真的按照预期运行，而不是只依赖一次 Demo 的展示效果。

工程思维转变：从功能实现到系统闭环

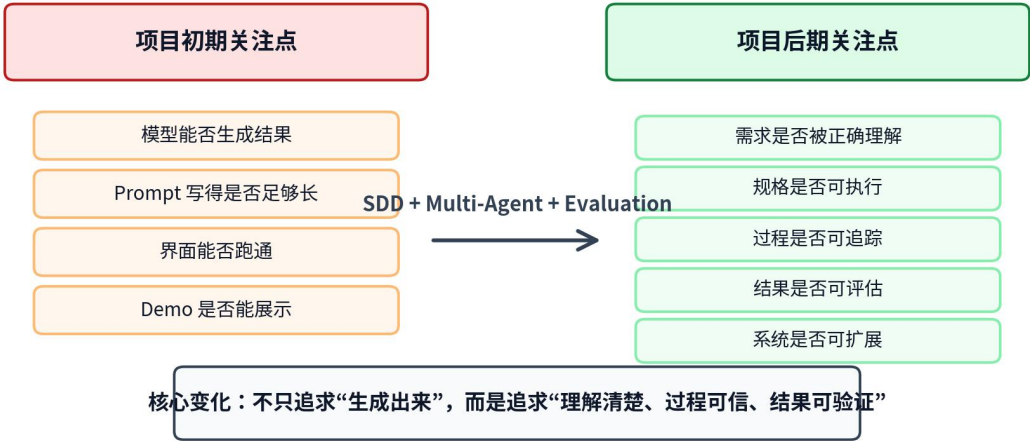


图 7-2 工程思维转变：从功能实现到系统闭环

7.4 对 Agentic AI 的理解

通过本项目，我对 Agentic AI 的理解也更加清晰。Agentic AI 并不只是“让大模型多次调用工具”，而是要围绕任务目标设计一套能够感知状态、规划步骤、调用工具、记录过程并根据结果做出判断的系统机制。Agent 的价值不只在于生成内容，还在于把复杂任务拆解成多个可控步骤。

在 Spec2Vision 中，用户输入的模糊 Prompt 并不会直接进入图像模型，而是先经过任务识别、需求澄清和 Visual Spec 生成。这个过程体现了 Agentic AI 的核心特点：系统能够根据当前信息判断下一步应该做什么。如果输入足够明确，系统可以继续

续生成；如果输入不完整，系统会先澄清；如果结果需要评估，系统会调用 `Evaluator` 输出评分。这种基于状态和任务目标的执行逻辑，比单次模型调用更加接近真实软件系统的需求。

此外，`MCP` 和 `Agentic RAG` 的集成让我理解到，`Agentic AI` 系统需要与外部工具和知识源建立清晰边界。`RAG` 使系统能够利用领域知识库补充视觉规范，`MCP` 则使系统能力可以被其他工具或 `Agent` 调用。这些能力说明，未来 `AI` 应用并不是孤立的聊天机器人，而是可以嵌入到更大软件生态中的智能组件。

7.5 不足与反思

虽然项目已经完成了核心功能，但仍然存在一些可以继续改进的地方。首先，当前系统的澄清问题主要基于规则和模板，虽然能够覆盖常见场景，但对于更复杂的视觉需求，仍然需要更灵活的动态问题生成能力。后续可以结合更强的 `LLM` 推理，让系统根据上下文自动生成更个性化的澄清问题。

其次，当前评估体系虽然包含确定性检查、启发式指标和 `Rubric` 评分，但视觉质量本身仍然具有一定主观性。对于真实生产场景，可能需要引入更多人工反馈、`VLM` 评估和用户偏好学习，使系统能够根据用户反馈逐步优化生成策略。

第三，当前项目主要面向课程演示和本地可复现运行，虽然已经加入 `Docker`、配置管理和测试，但在多用户权限、访问控制、成本监控、日志告警和在线部署稳定性方面仍有继续提升空间。这些内容也是系统从课程项目走向实际应用时必须考虑的问题。

7.6 课程学习体会

从学习体验来看，本课程对我最大的帮助，是让我把 `AI` 应用开发从“模型调用”提升到了“软件工程系统设计”的层面。过去我更容易把 `AI` 项目理解为调用大模型接口、写 `Prompt`、展示输出结果；但通过 `Proposal`、`Specs`、`MVP`、`Final Report` 等阶段性任务，我逐渐意识到，一个完整的 `AI` 应用不仅要能运行，还需要有清晰的问题定义、可执行的规格文档、合理的系统架构、稳定的工程实现和可信的测试评估。

在完成 `Spec2Vision` 的过程中，我对 `Agentic AI` 原生开发有了更具体的理解。`Agent` 并不是简单地把多个模型调用串联起来，而是要围绕任务目标进行职责拆分、状态传递、工具调用和结果反馈。尤其是在需求澄清、`Visual Spec` 生成、`Prompt` 构

造、资产生成和结果评估这些环节中，我逐渐体会到，多 Agent 系统的价值不仅在于增强模型能力，也在于让复杂任务变得更可解释、更可维护、更容易测试。

总体而言，这门课程让我更系统地理解了 AI 应用从需求分析到系统落地的完整过程，也让我意识到 AI 项目的质量不仅取决于模型本身，更取决于是否能够把模型能力放进一个清晰、稳定、可扩展的软件工程体系中。通过本项目，我对规格驱动开发、多 Agent 架构设计、工具封装、测试评估和工程化交付都有了更完整的认识。

7.7 本章小结

通过完成 Spec2Vision，我从最初关注“如何生成一张图”，逐步转向关注“如何设计一个可理解、可追踪、可评估、可扩展的 Agentic AI 系统”。这一过程让我对软件工程方法和 AI 应用开发都有了更深入的认识。

本项目最终实现了从模糊需求澄清到 Visual Spec 生成、从多 Agent 协作到结果评估、从本地 Demo 到工程化测试的完整流程。更重要的是，它让我认识到，好的 AI 应用不是简单地把模型能力包装成界面，而是要通过清晰的规格设计、合理的架构拆分和可靠的评估机制，把 AI 能力转化为可以被用户信任的软件系统。

总体来看，本项目使我在需求理解、规格设计、Agent 架构、工程实现、测试评估和课程反思等方面都有了系统性提升，也为后续继续深入学习 AI 工程化开发奠定了基础。